Build Long-Running Agents

The 6-Layer Architecture of an Agent That Waits for Days, Survives a Crash and Never Charges You Twice

Long-running agent = Trigger + Runner + State + Resumable Workflow + Human Gate + Guards.

Based on Google Cloud's Agent 102 "Long Running Agent" and Agent 101 "Agent Memory & GraphRAG" live workshops (Annie Wang, Google Cloud, October 2026), the AI harness crash course and the open-source Agent Development Kit (ADK Python 2.13).

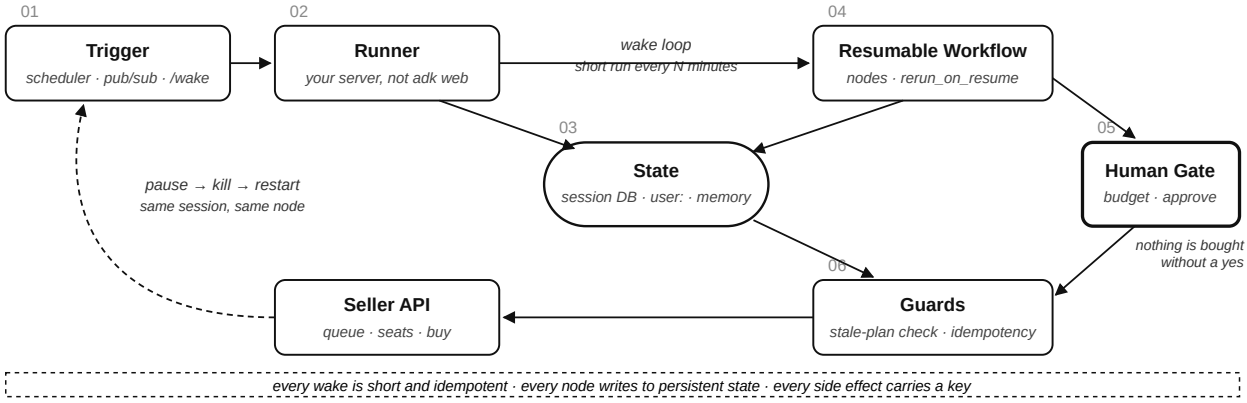


Fig. 1. Six layers around one persistent state. The trigger wakes a runner your server owns; the runner advances a workflow graph whose nodes record their outputs, so a run paused in a ticket queue survives a restart and resumes at the same node. Money moves only after the human gate, and only through guards that re-check live data and refuse a second charge. Layer names and framing are the author's.

Abstract—**Most agents live for one request. Ask one to "keep checking and tell me when it's my turn" and it will cheerfully agree, then do nothing, because nothing ever wakes it up again. Google Cloud's Agent 102 workshop builds the opposite: an agent that waits in a concert ticket queue for hours or days, survives its server being killed mid-task, pauses for a human to set a budget, re-checks a plan that went stale while it waited and makes sure a retried purchase never charges twice. This note distills the workshop and its companion memory session into six layers. For each it gives what the layer does, how the agent fails without it, the fix and working ADK 2.x code checked against the framework source; then a deployment map, a failure table, a five-step build plan, an 11-point checklist and a glossary. The thesis: waiting is not a model feature, it is infrastructure the model sits inside.**

Index Terms—long-running agents, resumable workflows, human-in-the-loop, agent memory, idempotency, Agent Development Kit, Cloud Run.

I. Why "Check Again Later" Never Happens

The failure every agent builder hits first

The workshop opens with a single agent and three tools: search events, get the seat map, purchase. Tell it the sale hasn't opened and ask it to check again in 10 seconds. It replies that it will keep watching. It never does.

> *"Even though it's giving the answer, it's not working... we really need a trigger." (Annie Wang, Agent 102)*

A model call is stateless and ends when the reply is sent. No instruction, however detailed, creates a timer. Waiting is an infrastructure problem: something outside the model has to wake the agent, and something has to remember where it was. The rest of this note builds both.

The fix is not a better prompt. The model decides what to do next; the code around it owns the clock, the database and the money.

TABLE I. WHAT EACH APPROACH TO "WAITING" ACTUALLY DOES

Approach	What happens	Verdict
"Check every 10 s" in the instruction	Agent agrees, schedules nothing, session ends	Fails silently
Loop inside one tool call	Blocks the run; dies with the process; burns tokens and quota	Fragile
External trigger → wake endpoint → runner	Each wake is a fresh, short run against saved state	Works
Trigger + resumable workflow + persistent DB	Waits days, survives restarts, resumes at the same node	Production

II. Layer 1: The Trigger

Something outside the model has to wake it up

What it is. A clock or event that sends a request to your app. The workshop starts with a one-shot timer ("fire once, 10 seconds from now"), then moves to a scheduled job in production. The trigger never talks to the model directly. It calls an HTTP endpoint, and the endpoint hands the work to the runner.

Listing 1. A wake endpoint that runs the agent against an existing session

```
from fastapi import FastAPI
from google.genai import types

api = FastAPI()

@api.post("/wake/{user_id}/{session_id}")
async def wake(user_id: str, session_id: str):
    msg = types.Content(role="user", parts=[types.Part(text="Wake-up: check the queue now.")])
    replies = []
    async for event in runner.run_async(user_id=user_id, session_id=session_id, new_message=msg):
        if event.content and event.content.parts:
            replies += [p.text for p in event.content.parts if p.text]
    return {"ok": True, "replies": replies}
```

Listing 2. The production trigger: a Cloud Scheduler job every 10 minutes

```
gcloud scheduler jobs create http ticket-watch \
  --schedule="*/10 * * * *" \
  --uri="https://ticket-agent-xyz.a.run.app/wake/u_123/s_abc" \
  --http-method=POST --location=us-central1
```

TABLE II. TRIGGER OPTIONS

Trigger	Use it for	Note
One-shot timer	"Check again in N seconds" during development	The workshop's first test
Cloud Scheduler (cron)	Polling a queue, a price, a status page	Calls the wake endpoint
Pub/Sub message	Reacting to an event (sale opened, payment settled)	Push subscription → endpoint
Webhook / UI click	A human or another system says "go"	Same endpoint, different caller

Failure without it: the agent's promise to follow up is a sentence, not a schedule. **Fix:** put the clock outside the model and make every wake a short, idempotent run.

III. Layer 2: Your Own Runner

adk web is a development tool, not a server

In ADK the runner is the event executor that powers the loop: it opens or loads a session, calls the model, runs tools, appends events and persists them. `adk web` ships a runner for you, which is why everything works in the browser and nothing works in production. A trigger needs a runner your server owns, wired to real storage.

Listing 3. A production runner: persistent sessions, memory and artifacts

```
from google.adk import Runner
from google.adk.apps import App, ResumabilityConfig
from google.adk.apps.app import EventsCompactionConfig
from google.adk.sessions import DatabaseSessionService
from google.adk.memory import VertexAiMemoryBankService
from google.adk.artifacts.gcs_artifact_service import GcsArtifactService

app = App(
    name="ticket_agent",
    root_agent=ticket_workflow,                      # Listing 7
    resumability_config=ResumabilityConfig(is_resumable=True),
    events_compaction_config=EventsCompactionConfig(compaction_interval=5, overlap_size=1),
)
runner = Runner(
    app=app,
    session_service=DatabaseSessionService(db_url="sqlite+aiosqlite:///./sessions.db"),
    memory_service=VertexAiMemoryBankService(project="my-proj", location="us-central1",
                                             agent_engine_id="1234567890"),
    artifact_service=GcsArtifactService(bucket_name="ticket-agent-artifacts"),
)
```

TABLE III. RUNNER VS HARNESS, AS CLARIFIED IN THE WORKSHOP Q&A

Concept	Scope
Runner	ADK's event executor: drives the reason → tool → event loop for one invocation
Harness	Everything around the model: context and memory, tool execution, workflow, sandboxing, privacy

IV. Layer 3: Persistent State

If it lives in RAM, it dies with the process

The workshop separates three things. **Events** record what happened, in order (the user asked for Amsterdam, the tool returned shows, the agent picked Sunday). **State** is the scratch pad of values that matter right now (party size 2, target event ID). **Memory** is distilled facts that should outlive the session (Sam can't do weeknights). Long-running work needs all three on disk.

TABLE IV. STATE SCOPES IN ADK

Prefix	Lives for	Example
(none)	This session	`target_event_id`, `queue_position`
`user:`	Every session of this user in this app	`user:seat_preference`, `user:budget_cap`
`app:`	All users of the app	`app:venue_rules`
`temp:`	The current invocation only, never persisted	`temp:raw_seat_map`

Listing 4. A tool that writes state, and an instruction that reads it

```
from google.adk.tools import ToolContext

def save_plan(event_id: str, party_size: int, tool_context: ToolContext) -> dict:
    tool_context.state["target_event_id"] = event_id       # session scope
    tool_context.state["party_size"] = party_size
    tool_context.state["user:last_city"] = "Amsterdam"      # follows the user
    return {"saved": True}

picker = Agent(
    name="pick_show",
    instruction="Book for {party_size?} people. Known preference: {user:seat_preference?}.",
    tools=[search_events, get_seat_map, save_plan],
)
```

TABLE V. SESSION SERVICES: WHERE THE TRANSCRIPT LIVES

Service	Survives a restart	Best for
InMemorySessionService	No	Tests, voice agents where speed matters
DatabaseSessionService (SQLite)	Yes	Local and small projects; the workshop default
DatabaseSessionService (Cloud SQL / Postgres)	Yes	Production, many instances
Vertex AI / Agent Engine sessions	Yes	Fully managed deployments

Memory: the four types, and the two that matter here

TABLE VI. AGENT MEMORY TYPES (AGENT 101: MEMORY & GRAPHRAG)

Type	Question it answers	ADK building block
Working	What are we doing right now?	Session + state
Episodic	What happened before?	Persisted sessions
Semantic	What do I know about this user?	Memory service (Memory Bank)
Procedural	How do I do this task?	Instructions, skills

TABLE VII. MEMORY SERVICES COMPARED

Service	Write	Read
InMemoryMemoryService	Stores whole events, no distilling	Keyword match only
VertexAiMemoryBankService	Gemini extracts facts and consolidates them ("blue, not purple anymore")	Semantic search by meaning
Custom (the Agent 102 lab uses a Markdown store)	Your rules	Your rules

Listing 5. Recall at the start, remember at the end

```
from google.adk.tools import preload_memory   # injects relevant memories before each turn

picker = Agent(name="pick_show", instruction="...", tools=[search_events, save_plan, preload_memory])

# after a session worth keeping, distill it into long-term memory
session = await session_service.get_session(app_name="ticket_agent", user_id="u_123", session_id="s_abc")
await memory_service.add_session_to_memory(session)
```

Long conversations. Turn on compaction (Listing 3): every *N* invocations ADK summarizes older events with an overlap, the same way coding agents compact a long thread, so a days-long session never overflows the context window.

V. Layer 4: The Resumable Workflow

Kill the server mid-queue. It picks up at the same node.

The workshop's centerpiece: the agent joins the queue, pauses, and the presenter kills the process. On restart the same session loads from the database and the run continues from where it stopped, with every preference intact. Two things make this work: a **workflow graph** whose nodes record their outputs, and a **resumable app**.

Listing 6. A node that waits for the front of the queue and re-runs on every resume

```
from google.adk import Context, Event
from google.adk.events import RequestInput
from google.adk.workflow import node

@node(rerun_on_resume=True)
def check_front(ctx: Context):
    status = seller.queue_status(ctx.state["queue_ticket"])   # fresh check on every wake
    if status["position"] > 0:
        yield Event(state={"queue_position": status["position"]})
        yield RequestInput(interrupt_id=f"queue_{ctx.invocation_id}",
                           message=f"Position {status['position']}. Waiting for the next wake-up.")
        return
    yield Event(state={"at_front": True}, route="ready")
```

Listing 7. The ticket workflow as a graph

```
from google.adk import Workflow

ticket_workflow = Workflow(
    name="ticket_agent",
    edges=[
        ("START", pick_show, queue_up, check_front),
        (check_front, {"ready": budget_gate}),
        (budget_gate, {"approved": purchase, "changed": budget_gate}),
    ],
)
```

TABLE VIII. WHAT HAPPENS TO EACH NODE ON RESUME

Node	On resume	Why
pick_show, queue_up (finished)	Recorded output is reused, not called again	No duplicate searches or queue joins
check_front (`rerun_on_resume=True`)	Runs again	Queue position is stale the moment you pause
budget_gate (`rerun_on_resume=False`)	The human's answer becomes the node output	Don't re-ask a question already answered

TABLE IX. TWO WAYS TO REPRESENT A LONG WAIT

Mechanism	Use when	Resumes with
`LongRunningFunctionTool`	A job runs elsewhere (video render, batch, payment settlement)	A function response sent back with the same call ID
Workflow node + `RequestInput`	A human or an external signal must answer	`ctx.resume_inputs[interrupt_id]` on the next run

Listing 8. Wrapping a slow external job, and resuming a stopped invocation

```
from google.adk.tools.long_running_tool import LongRunningFunctionTool

def request_refund(order_id: str) -> dict:
    ticket = billing.open_refund(order_id)          # returns immediately
    return {"status": "pending", "ticket": ticket}

support = Agent(name="support", instruction="...", tools=[LongRunningFunctionTool(func=request_refund)])

# resume an invocation that was paused or failed midway
async for event in runner.run_async(user_id="u_123", session_id="s_abc", invocation_id="inv-42"):
    print(event)
```

> **The foundation.** Resumability is only as good as storage. With an in-memory session service a restart wipes the queue position, the plan and the user's preferences. The persistent database from Layer 3 is what turns a pause into a resume.

VI. Layer 5: The Human Gate

The agent spends money only inside a budget a human set

Before buying, the workshop's agent stops and asks: how much per ticket? It reads the answer ("two tickets, up to $250 each"), restates it, and waits for a yes. The pause survives restarts like any other node.

Listing 9. Budget gate: ask, confirm, then route

```
@node(rerun_on_resume=True)
def budget_gate(ctx: Context):
    key = f"budget_{ctx.invocation_id}"
    answer = ctx.resume_inputs.get(key)
    if not answer:
        yield RequestInput(interrupt_id=key,
                           message="Two seats found. Max price per ticket? (e.g. 'up to $250 each')")
        return
    cap = parse_money(answer)                       # your parser
    yield Event(state={"user:budget_cap": cap, "approved_qty": 2}, route="approved")
```

TABLE X. WHAT SHOULD ALWAYS STOP FOR A HUMAN

Action	Gate
Spending money above zero	Budget set and confirmed by the user
Changing a confirmed plan (seat, date, quantity)	Re-ask, never assume
Anything that can't be undone (send, publish, delete)	Explicit yes in the same session

VII. Layer 6: Guards

Memory remembers the old plan perfectly. That's the problem.

The plan was valid when it was saved. Hours later, in a queue shared with thousands of buyers, the seat may be gone or the price may have moved. The workshop adds two guards in front of the purchase tool: a **before-tool callback** that re-validates the plan against live data, and an **idempotency key** so a retry after a timeout or a 503 can never become a second charge.

Listing 10. A before-tool callback that blocks a stale or over-budget purchase

```
from typing import Any, Optional
from google.adk.tools import BaseTool, ToolContext

def purchase_guard(tool: BaseTool, args: dict[str, Any], tool_context: ToolContext) -> Optional[dict]:
    if tool.name != "purchase":
        return None                                          # let other tools run
    s = tool_context.state
    live = seller.get_seat(args["seat_id"])                  # fresh data, not the saved plan
    if not live["available"]:
        return {"error": "seat_taken", "next": "pick another seat within budget"}
    if args["quantity"] != s.get("approved_qty"):
        return {"error": "quantity_changed_without_approval"}
    if args["currency"] != "USD" or live["price"] > s.get("user:budget_cap", 0):
        return {"error": "over_budget", "price": live["price"]}
    return None                                              # returning a dict skips the tool

buyer = Agent(name="purchase", instruction="...", tools=[purchase], before_tool_callback=purchase_guard)
```

Listing 11. An idempotent purchase: one intended order, however many retries

```
def purchase(seat_id: str, quantity: int, currency: str, tool_context: ToolContext) -> dict:
    key = tool_context.state.get("order_key")
    if not key:
        key = f"{tool_context.session.id}:{seat_id}:{quantity}"
        tool_context.state["order_key"] = key                # persisted before the call
    res = seller.buy(seat_id=seat_id, qty=quantity, idempotency_key=key)
    tool_context.state["order_id"] = res["order_id"]         # same key → same order
    return res
```

TABLE XI. THE THREE GUARDS

Guard	Stops	Where
Human budget gate	Spending the user never agreed to	Workflow node (Layer 5)
Stale-plan check	Buying a seat that's gone or now over budget	`before_tool_callback`
Idempotency key	A retried request charging twice	Inside the tool + seller API

Why it matters on resume. ADK resumes on a best-effort basis and a tool may run more than once. Any tool with side effects (purchases, emails, refunds) must check for a previous run before acting.

VIII. Shipping It on Google Cloud

The same graph, with managed pieces underneath

TABLE XII. LOCAL LAB → PRODUCTION

Piece	In the lab	In production
Agent server	adk web / local FastAPI	Cloud Run
Trigger	One-shot timer	Cloud Scheduler, Pub/Sub push
Sessions	SQLite file	Cloud SQL (Postgres)
Memory	Markdown / in-memory	Vertex AI Memory Bank or RAG memory
Artifacts	Local folder	Cloud Storage

Listing 12. Deploy the server and point the trigger at it

```
gcloud run deploy ticket-agent --source . --region us-central1 \
  --set-env-vars DB_URL="postgresql+asyncpg://agent:PASS@/tickets?host=/cloudsql/PROJ:us-central1:agent-
db"
gcloud scheduler jobs update http ticket-watch \
  --uri="$(gcloud run services describe ticket-agent --region us-central1 --
format='value(status.url)')/wake/u_123/s_abc"
```

IX. Failure Modes and Fixes

TABLE XIII. WHAT BREAKS, AND THE LAYER THAT FIXES IT

Symptom	Cause	Fix
"I'll keep watching" and nothing happens	No trigger	Layer 1
Works in adk web, dead on the server	No runner of your own	Layer 2
Deploy or crash loses the queue spot	In-memory sessions	Layer 3
Agent forgets the user's preferences next week	Session-only state	`user:` state + Memory Bank
Resume re-joins the queue or re-asks the user	No recorded node outputs	Workflow + correct `rerun_on_resume`
Context window overflows after days	Unbounded transcript	Event compaction
Buys a seat someone else just took	Stale plan	`before_tool_callback`
Customer charged twice after a timeout	Non-idempotent tool	Idempotency key

X. Build Plan, Checklist and Glossary

TABLE XIV. A FIVE-STEP BUILD PLAN

Step	Build	Done when
1	Single agent + tools in adk web	It finds the event and the seats
2	Own runner + wake endpoint + one-shot timer	A timer wakes it and it reports back
3	DatabaseSessionService + `user:` state + memory	Kill and restart: preferences are still there
4	Workflow graph + RequestInput + ResumabilityConfig	Kill mid-queue: it resumes at check_front
5	Budget gate + purchase guard + idempotency, then Cloud Run	A forced retry produces one order

Pre-launch checklist

☐ Trigger lives outside the model
☐ Every wake is short and idempotent
☐ Sessions in a real database
☐ Preferences in `user:` state or memory
☐ Long waits are workflow nodes, not loops
☐ Live checks re-run on resume
☐ Human answers are not re-asked
☐ Spending needs a confirmed budget
☐ Purchase re-validates live data
☐ Side-effect tools carry an idempotency key
☐ Compaction on for multi-day sessions

TABLE XV. GLOSSARY

Runner	ADK's event executor for one invocation
Session	The ordered events of one conversation
State	Key-value scratch pad shared by nodes
Memory	Facts that outlive a session
Interrupt	A RequestInput that pauses a node
Resume	Continuing an invocation from its last event
Idempotency key	One ID per intended action, reused on retry

References

[1] A. Wang, "Agent 102: Long Running Agent," Google Cloud live workshop, Oct. 2026 (lab design by Christina, Google Cloud).

[2] A. Wang, "Agent 101: Agent Memory & GraphRAG," Google Cloud live workshop, 2026.

[3] Google, "AI harness crash course," Google Cloud Tech, Oct. 2026.

[4] Google, Agent Development Kit for Python, v2.13: Workflow, RequestInput, ResumabilityConfig, EventsCompactionConfig, DatabaseSessionService, LongRunningFunctionTool, and the *request_input_rerun* sample. github.com/google/adk-python.

[5] Google, "Resume stopped agents," ADK documentation.

Method: compiled from the workshop recordings; seller API calls (queue_status, get_seat, buy) are illustrative stand-ins for the lab's ticket service. All ADK class and parameter names were checked against the ADK Python 2.13 source. Independently compiled; not affiliated with or endorsed by Google.